Automated Brute-Force Detection & Incident Analysis

A Practical SOC Lab Implementation with Splunk SIEM

Project Name	Automated Brute-Force Detection & Incident Analysis
Team Name	_____ Group 4 _____
Core Platform	Splunk Enterprise (SIEM)
Date	August 2026
Team Members	Mohanned Ahmed Ali Arafa Abdelrahman Mahmoud Ali Ahmed (team leader) Mahmoud Ali Elsayed Elesawy Mahmoud Alaa Eldin Abdulrahman Ayesh Abdullah Mohamed Abdullah Awaad _____

Tools & Technologies:

Kali Linux • Ubuntu Server • OpenSSH • rsyslog • Splunk Enterprise (SIEM) • THC-Hydra

Table of Contents

Table of Contents 2
1. Introduction 5
1.1 Motivation 5
1.2 Project Objectives 5
2. Project Overview 5
2.1 Project Description 5
2.2 SOC Lab Architecture 6
2.3 Lab Environment 6
2.4 Attack Scenario 6
3. Infrastructure and Components 7
Kali Linux 7
Ubuntu Server 7
OpenSSH (openssh-server) 7
Hydra (THC-Hydra) 7
rsyslog 8
Syslog over UDP 514 8
Splunk Enterprise 8
4. Lab Configuration 8
4.1 Network Configuration 8
4.2 SSH Configuration 9
4.3 rsyslog Configuration 10
4.4 Splunk Data Input Configuration 11
5. Attack Simulation 15
5.1 Attack Objective 15
5.2 Hydra 15
5.3 Password List 15
5.4 SSH Authentication Attempts 16
Stage 1 — Attack with an incorrect password list 16
Stage 2 — Attack with the correct password included 17
Post-Compromise Access 17
6. Log Collection and SIEM Integration 18
7. Detection and Incident Analysis 19
7.1 Detection Logic 19
7.2 SPL Queries 19
Query 1 — Initial brute-force and successful-login detection 19

Query 2 — Updated query after post-authentication folder activity 20
7.3 Successful Authentication 21
7.4 Post-Authentication Activity 21
8. Incident Investigation Workflow 22
9. Splunk Dashboard 22
10. Challenges and Solutions 24
11. Testing and Validation 25
12. Results and Findings 26
Technical Results 26
Security Results 26
SOC Value 26
13. Security Analysis 26
14. Future Improvements 27
15. Conclusion 27
16. References 28

1. Introduction

1.1 Motivation

Authentication abuse remains one of the most common initial-access techniques observed in real-world security incidents. SSH, in particular, is a high-value target because it provides direct, privileged shell access to a host once valid credentials are obtained. When authentication endpoints are exposed without adequate monitoring, an attacker can attempt large numbers of password guesses against a service with little risk of early detection, especially in environments where authentication logs are generated locally but never centrally collected, correlated, or reviewed.

From a Security Operations Center (SOC) perspective, the ability to detect brute-force authentication attempts — and to distinguish a blocked attack from a successful compromise — is a foundational monitoring capability. A SOC that cannot see failed and successful authentication events in one place, correlate them by source, and follow up with what an attacker did after gaining access, is operating with a significant blind spot. This project was built to demonstrate, hands-on, how that blind spot is closed using log forwarding and SIEM correlation.

1.2 Project Objectives

- Build a controlled, three-machine SOC lab consisting of an attacker system, a target SSH server, and a centralized SIEM.
- Configure centralized log forwarding from a Linux target host to Splunk using rsyslog over the Syslog protocol (UDP).
- Configure Splunk to receive and index the forwarded logs through a UDP network data input.
- Simulate a realistic SSH brute-force attack against the target host using Hydra with a custom password wordlist.
- Use Splunk Search Processing Language (SPL) to detect brute-force authentication patterns from the ingested logs.
- Investigate the incident end-to-end: identify failed login attempts, confirm successful authentication, and examine post-authentication activity performed by the attacker.
- Build a Splunk dashboard that visualizes the detection query so the investigation can be monitored on an ongoing basis.

2. Project Overview

2.1 Project Description

This project is a practical Security Operations Center (SOC) lab that demonstrates automated brute-force detection and incident analysis using Splunk as the SIEM platform. A Kali Linux attacker machine is used to simulate an SSH brute-force attack against an Ubuntu server. Authentication and system activity generated on the Ubuntu server is forwarded in real time to Splunk via rsyslog over Syslog/UDP. Once the logs are centralized in Splunk, SPL queries are used to detect the brute-force pattern, confirm whether the attack resulted in a successful compromise, and identify the actions the attacker carried out after gaining access. The findings are then pinned to a Splunk dashboard for ongoing visualization.

2.2 SOC Lab Architecture

The lab is built around a simple but complete SOC data pipeline: an attacker generates authentication traffic against a monitored target, the target's local logs are forwarded to a centralized SIEM, and the SIEM is used to detect and investigate the resulting activity. Conceptually, the architecture is represented as follows:

Kali Linux (Attacker) ↓
SSH Brute-Force Simulation (Hydra) ↓
Ubuntu SSH Server (Target) ↓
Authentication / System Logs (auth.log, cron, sudo) ↓
rsyslog (Log Forwarding Agent) ↓
Syslog over UDP 514 ↓
Splunk SIEM (Indexing & Search) ↓
SPL Search & Correlation ↓
Detection (Brute-Force Pattern) ↓
Incident Investigation ↓
Splunk Dashboard

Figure 1 — Conceptual SOC lab data and detection pipeline.

2.3 Lab Environment

The lab consists of three virtual machines, each with a defined role and a static IP address on the same local network segment, as documented in the project source material:

Machine	Role	IP Address	Purpose
Kali Linux	Attacker	192.168.1.175	Simulates the SSH brute-force attack using Hydra
Ubuntu Server	Target	192.168.1.133	Hosts the SSH service and generates authentication/system logs
Splunk Server	SIEM / Log Collector	192.168.1.17	Receives forwarded logs via UDP 514 and hosts detection queries and the dashboard

Table 1 — Lab environment and IP addressing.

2.4 Attack Scenario

The simulated scenario represents a common real-world security incident: an external actor (Kali) attempts to gain unauthorized access to a Linux server (Ubuntu) by systematically guessing SSH credentials using a dictionary/wordlist-based brute-force tool (Hydra). The scenario is executed in two stages to reflect a realistic attack progression. In the first stage, the attacker attempts a wordlist that does not contain the correct password, resulting in a blocked/failed attack. In the second stage, the attacker attempts an updated wordlist that does contain the correct password, resulting in a successful compromise. After gaining access, the attacker performs post-authentication actions (creating and deleting directories) that are also captured in the logs and investigated as part of the incident.

3. Infrastructure and Components

Each component in the lab plays a specific role in either generating, transporting, or analyzing security-relevant data. The table below summarizes the components used, followed by a detailed explanation of each.

Component	Role	Purpose
Kali Linux	Attacker platform	Runs Hydra to simulate SSH brute-force authentication attempts
Ubuntu Server	Target / log source	Runs the SSH service under attack and generates the authentication/system logs
OpenSSH (openssh-server)	Target service	Provides the SSH authentication service that is the object of the attack
Hydra (THC-Hydra)	Attack tool	Performs automated, dictionary-based SSH password guessing
rsyslog	Log forwarding agent	Forwards local Ubuntu logs to the Splunk server over the Syslog protocol
Syslog / UDP 514	Transport protocol	Carries forwarded log messages from Ubuntu to Splunk
Splunk Enterprise	SIEM platform	Indexes the forwarded logs and provides SPL search, correlation, and dashboarding

Kali Linux

Kali Linux is a penetration-testing-focused Linux distribution and is used in this lab as the attacker machine (IP 192.168.1.175). It hosts the Hydra tool used to launch the SSH brute-force attack against the Ubuntu target.

Ubuntu Server

Ubuntu Server (IP 192.168.1.133) is the target machine in the lab. It runs the OpenSSH service that is under attack, and it is the source of the authentication and system logs (generated through PAM/sudo/cron activity) that are forwarded to Splunk for analysis.

OpenSSH (openssh-server)

OpenSSH is the SSH server software installed on the Ubuntu target to provide remote shell access. It is the specific service targeted by the brute-force attack, and it is the component whose authentication events (success and failure) generate the log data that is later forwarded and analyzed in Splunk.

Hydra (THC-Hydra)

Hydra is a fast, parallelized network login-cracking tool. In this lab it is used from Kali Linux to perform automated, dictionary-based password-guessing attempts against the SSH service on the Ubuntu target, using a locally created password wordlist.

rsyslog

rsyslog is the system logging daemon running on the Ubuntu target. In its default configuration it only writes log messages to local files. In this project, rsyslog is reconfigured to additionally forward every log message to the Splunk server over the network, which is what allows the authentication and system events generated on Ubuntu to be centrally collected and analyzed rather than remaining isolated on the target host.

Syslog over UDP 514

Syslog is the standard protocol used for transmitting log messages between the Ubuntu host and the Splunk server. In this lab it is carried over UDP on port 514, which is the destination configured on the rsyslog side and the listening input configured on the Splunk side.

Splunk Enterprise

Splunk Enterprise is the SIEM platform used to receive, index, search, and visualize the forwarded logs. It is configured with a UDP network input on port 514 to receive the Syslog traffic from Ubuntu, and it is where the detection queries (SPL) and the investigation dashboard are built and run.

4. Lab Configuration

4.1 Network Configuration

All three machines were placed on the same local network segment so that the attacker (Kali) can reach the target (Ubuntu) over SSH, and the target (Ubuntu) can reach the SIEM (Splunk) over UDP. The virtual machine network adapter for the environment was configured in Bridged (Advanced) mode, using the host's Wi-Fi interface as the bridged interface and an emulated Intel Gigabit Ethernet (e1000) network card, so that each virtual machine receives an address on the same physical network.

Parameter	Value	Purpose
Network Mode	Bridged (Advanced)	Places the virtual machines on the same network segment as the physical host
Bridged Interface	Wi-Fi (en0)	Physical interface used to bridge the virtual machine network
Emulated Network Card	Intel Gigabit Ethernet (e1000)	Virtual NIC model presented to the guest OS
Ubuntu (target) IP	192.168.1.133	Address of the SSH server under attack
Splunk (SIEM) IP	192.168.1.17	Address the logs are forwarded to and the SIEM is accessed at
Kali (attacker) IP	192.168.1.175	Address the brute-force attack originates from

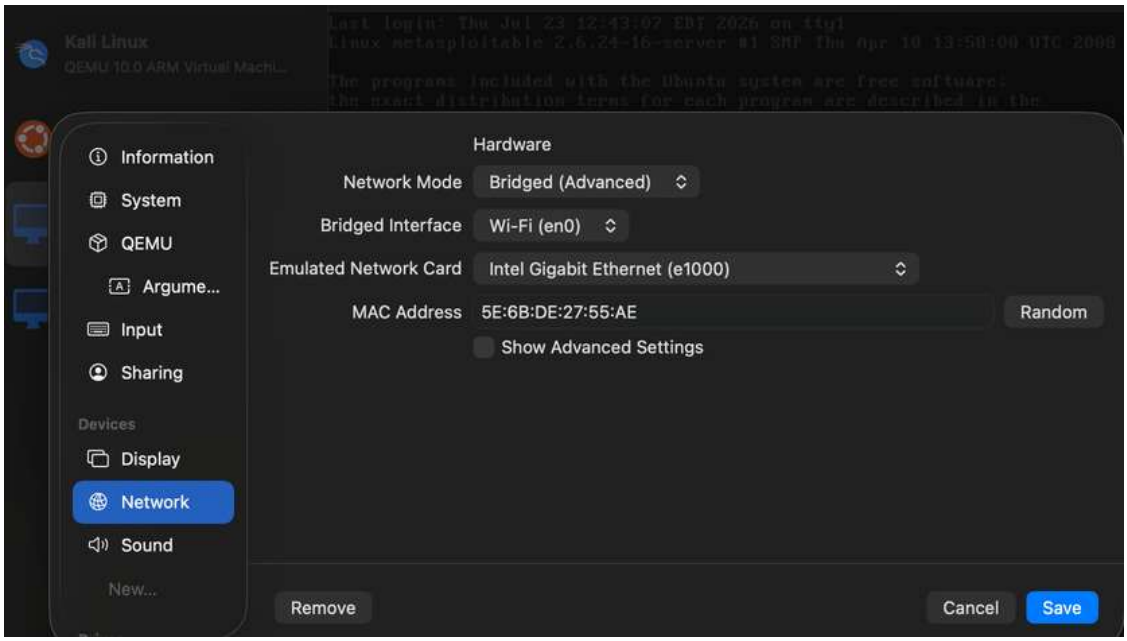


Figure 2 — Virtual machine network adapter set to Bridged (Advanced) mode over the Wi-Fi interface, with an emulated Intel Gigabit Ethernet (e1000) card.

4.2 SSH Configuration

On the Ubuntu target, the OpenSSH server package was installed and the SSH service was enabled so that it starts automatically and is immediately active, making the machine reachable for SSH authentication attempts:

```
sudo apt update && sudo apt install -y openssh-server
sudo systemctl enable --now ssh
```

The first command updates the local package index and installs the openssh-server package. The second command enables the SSH service to start on boot and immediately starts it in the same step (--now), so the target is ready to accept authentication attempts without a reboot. This is the service that Hydra later targets from the Kali machine.

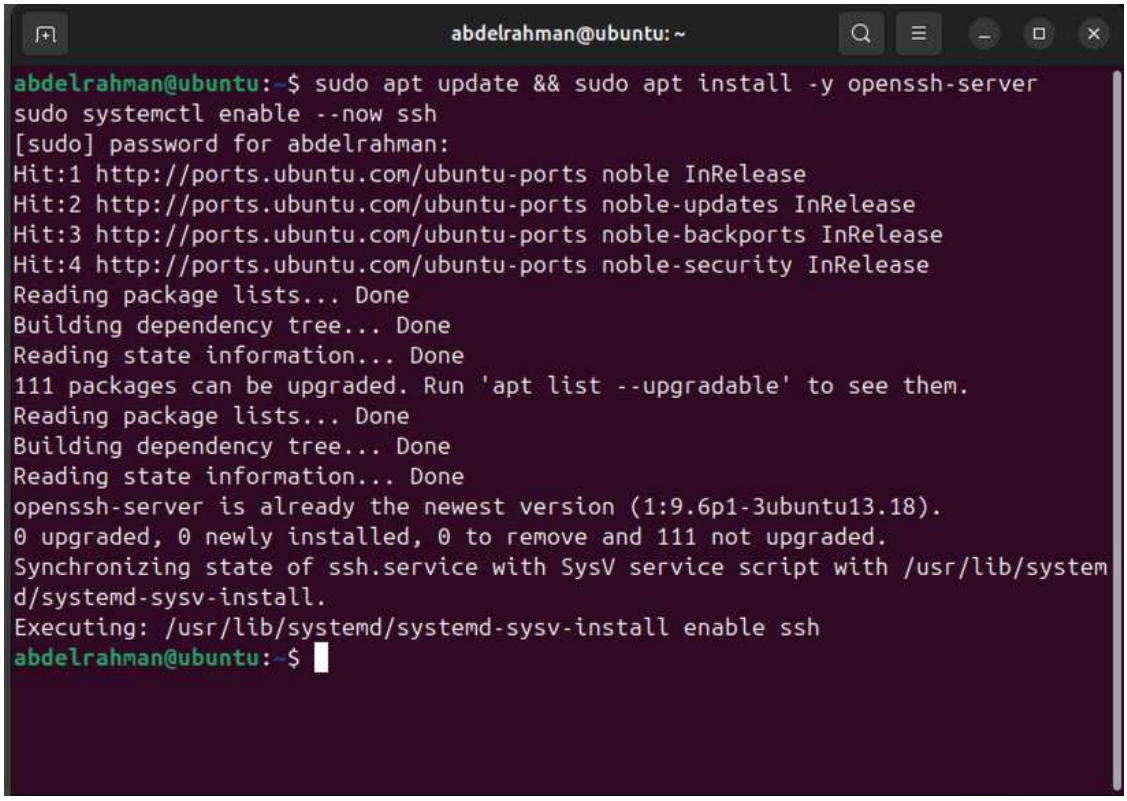


Figure 3 — Ubuntu terminal confirming installation and enabling of openssh-server.

4.3 rsyslog Configuration

rsyslog is responsible for forwarding the logs generated on the Ubuntu target to the Splunk server. Its configuration file was edited directly:

```
sudo nano /etc/rsyslog.conf
```

A forwarding rule was added at the end of the configuration file, directing all log facilities and severities (*.*) to the Splunk server's IP address over UDP on port 514:

```
*.* @192.168.1.17:514
```

The single "@" prefix specifies UDP as the transport (a double "@@" would indicate TCP). This line instructs rsyslog to relay every local log message it processes to the Splunk host at 192.168.1.17 on port 514, which is the same port later configured as a listening UDP input in Splunk.

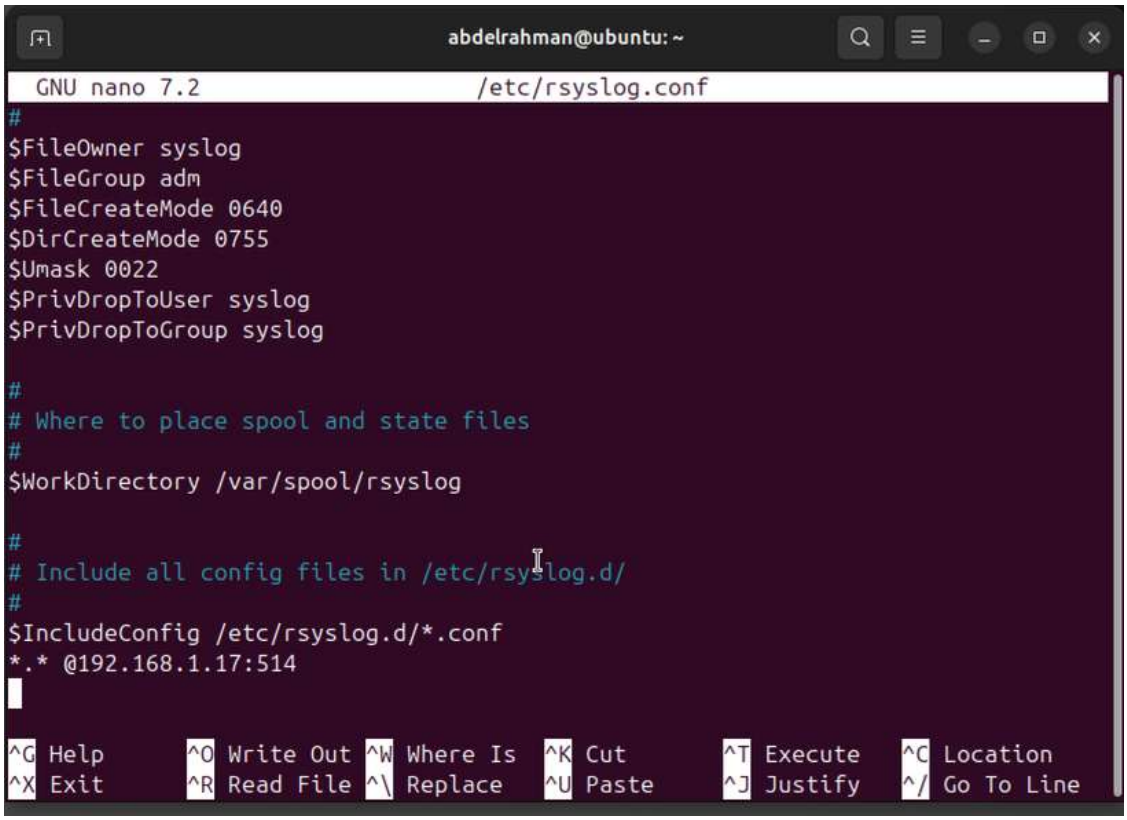


*Figure 4 — Editing /etc/rsyslog.conf with nano, showing the forwarding rule *.* @192.168.1.17:514 added at the end of the file.*

After saving the configuration change, the rsyslog service was restarted to apply it:

```
sudo systemctl restart rsyslog
```

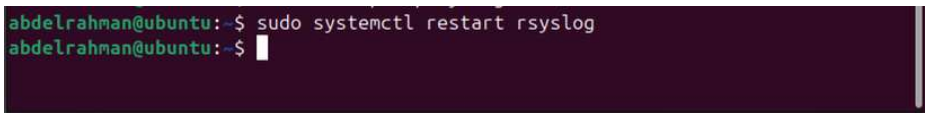


Figure 5 — Restarting the rsyslog service to apply the new forwarding configuration.

4.4 Splunk Data Input Configuration

On the Splunk server, a new local UDP data input was created to listen for the Syslog traffic forwarded by Ubuntu. The input was configured through the Splunk web interface as follows:

- Navigate to Settings > Data Inputs > UDP > New Local UDP.
- Set the listening Port to 514.
- Select the Syslog source type (Operating System category) for the incoming data.
- Review the configuration and Submit to create the input.

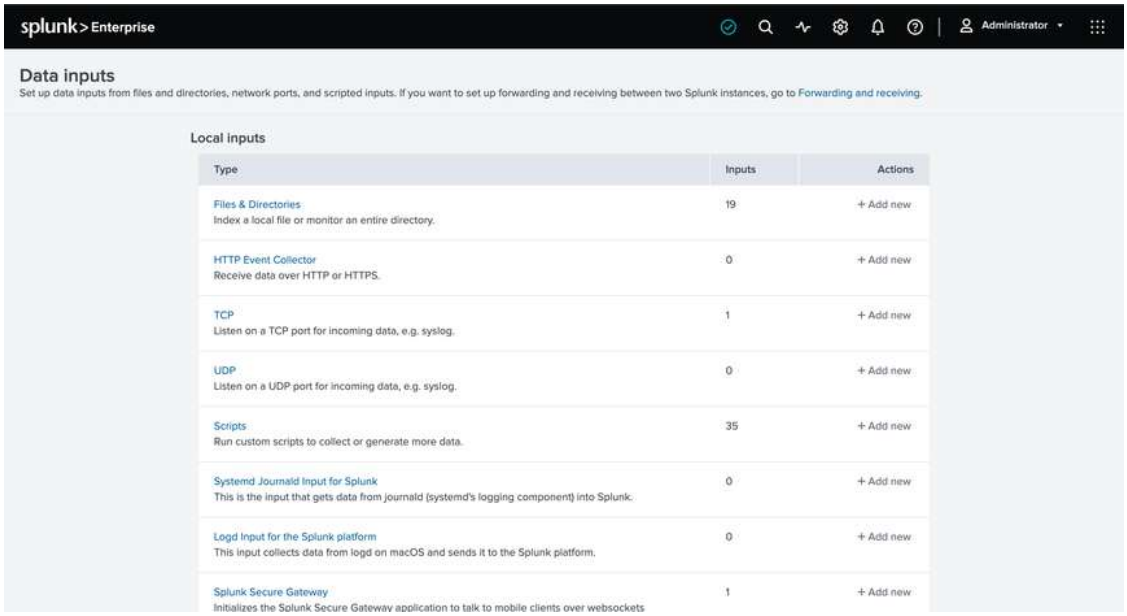


Figure 6 — Splunk Settings > Data Inputs, showing the available local input types including UDP.

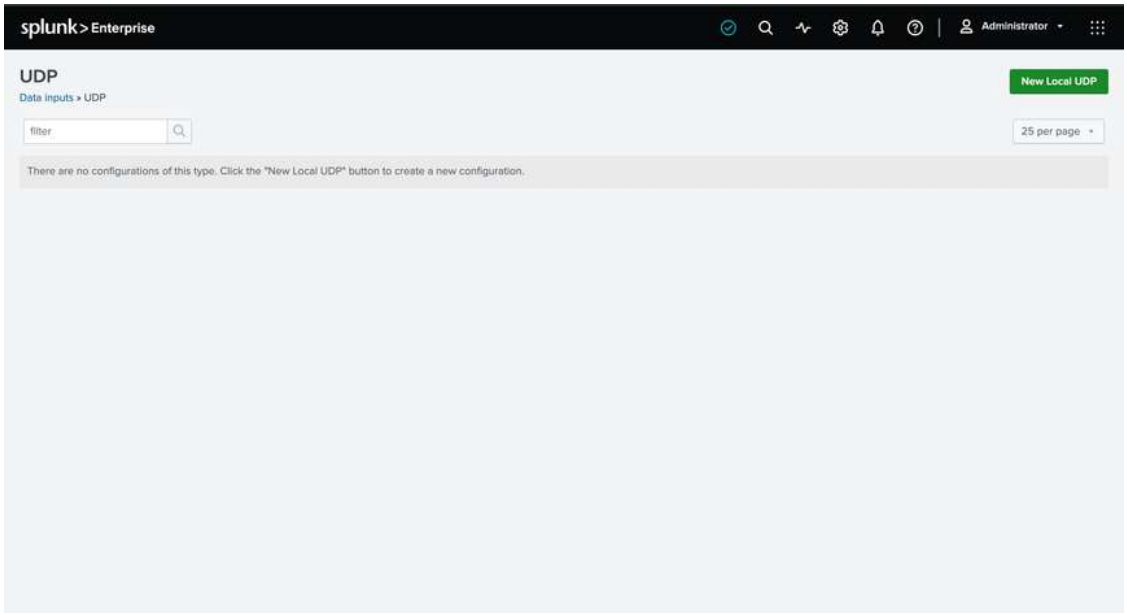


Figure 7 — UDP data inputs list before configuration — no inputs exist yet.

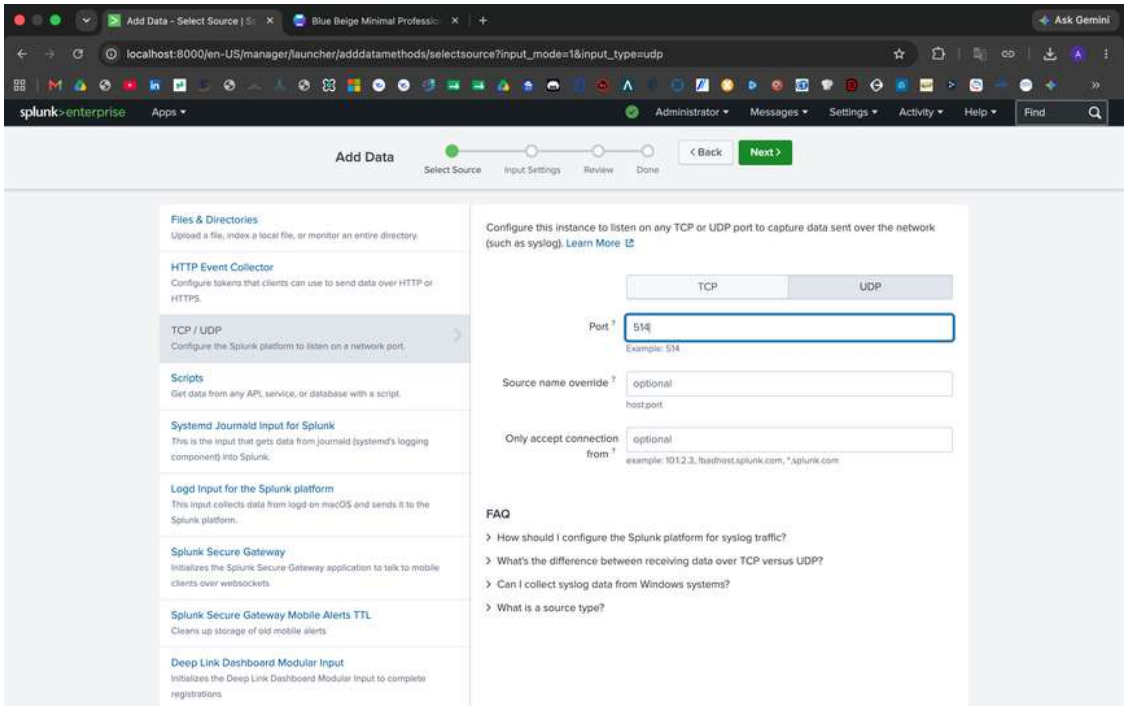


Figure 8 — Add Data wizard: configuring a new UDP input on port 514.

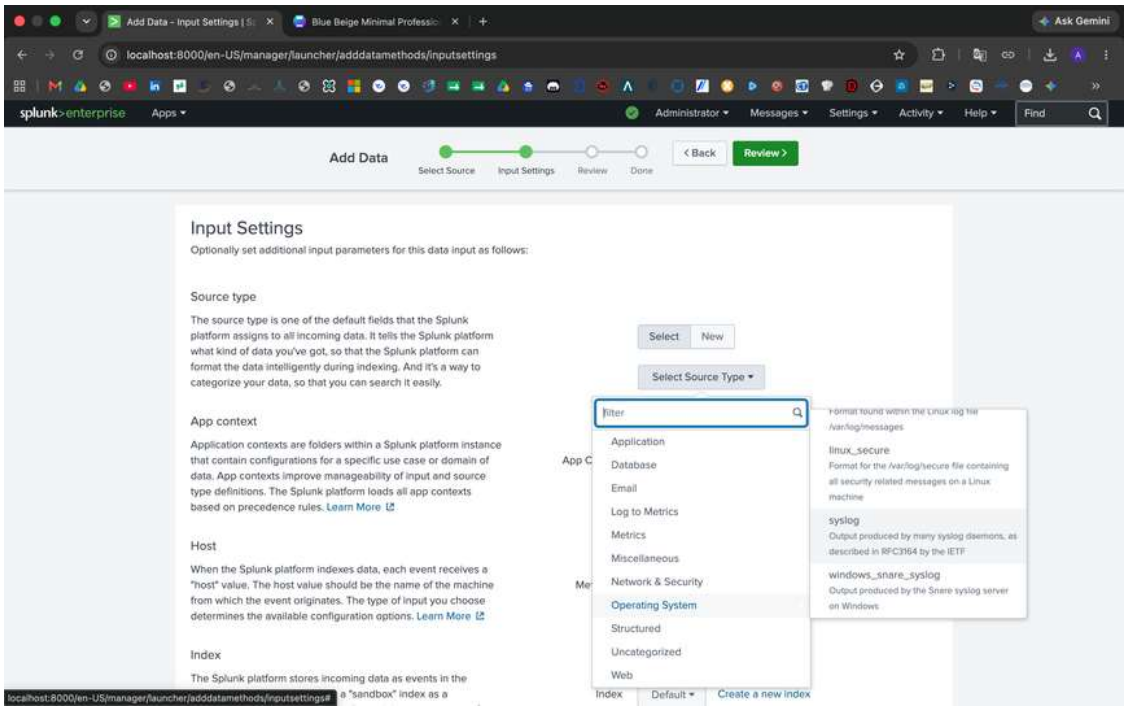


Figure 9 — Add Data wizard: selecting the syslog source type under the Operating System category.

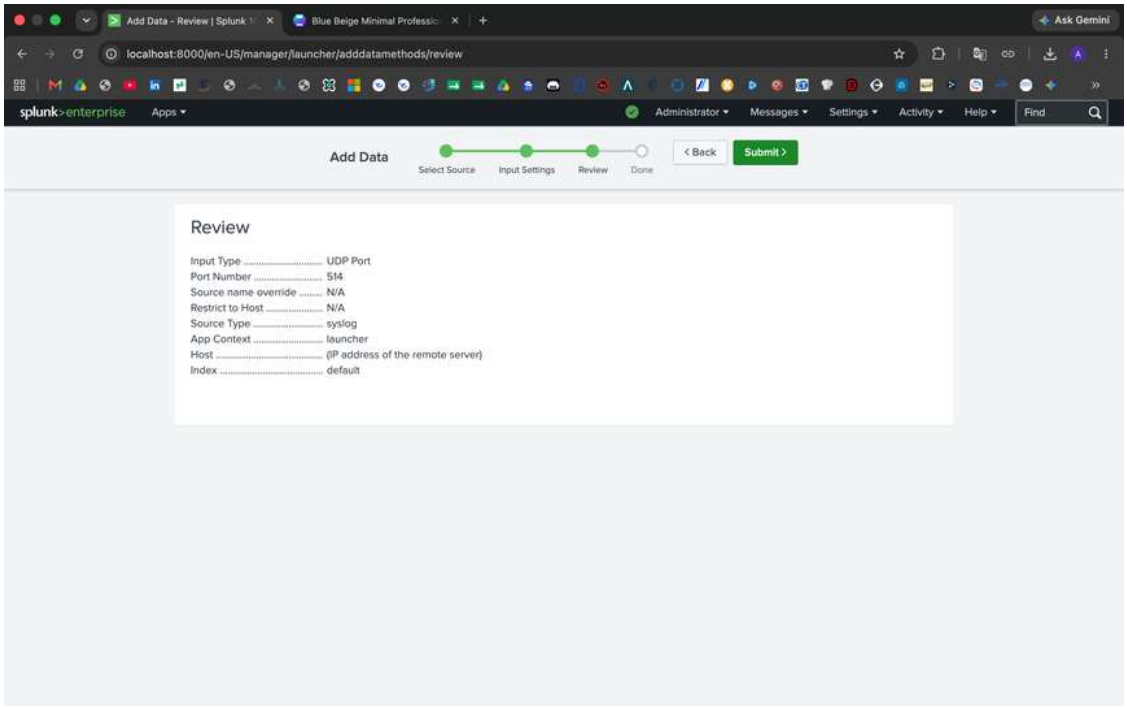


Figure 10 — Add Data wizard: review screen confirming Port 514, source type syslog, and index default.

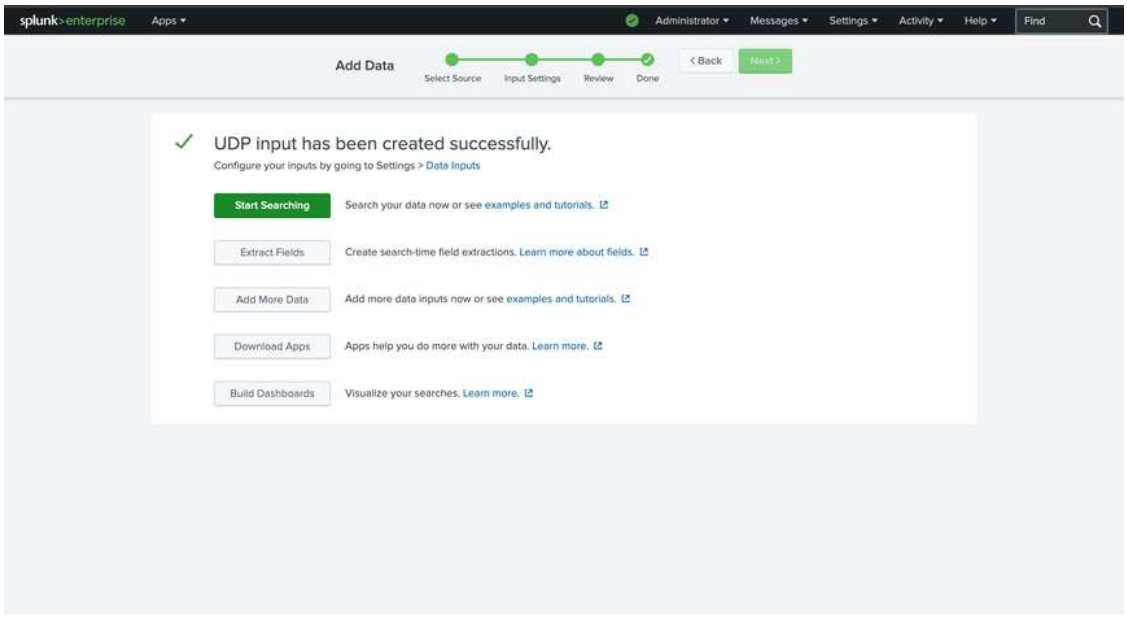


Figure 11 — Confirmation that the UDP input was created successfully.

The review step of the wizard confirmed the following input settings before creation:

Setting	Value
Input Type	UDP Port
Port Number	514
Source Type	syslog
App Context	launcher

Setting	Value
Host	IP address of the remote server
Index	default

Once submitted, Splunk confirmed: “UDP input has been created successfully,” and the platform began indexing the incoming Syslog data under sourcetype=syslog on source="udp:514". Searching source="udp:514" sourcetype="syslog" immediately after configuration returned 1,216 events, confirming that logs generated on Ubuntu — including PAM session events (pam_unix(cron:session)) and cron command executions — were being received and indexed by Splunk in real time.

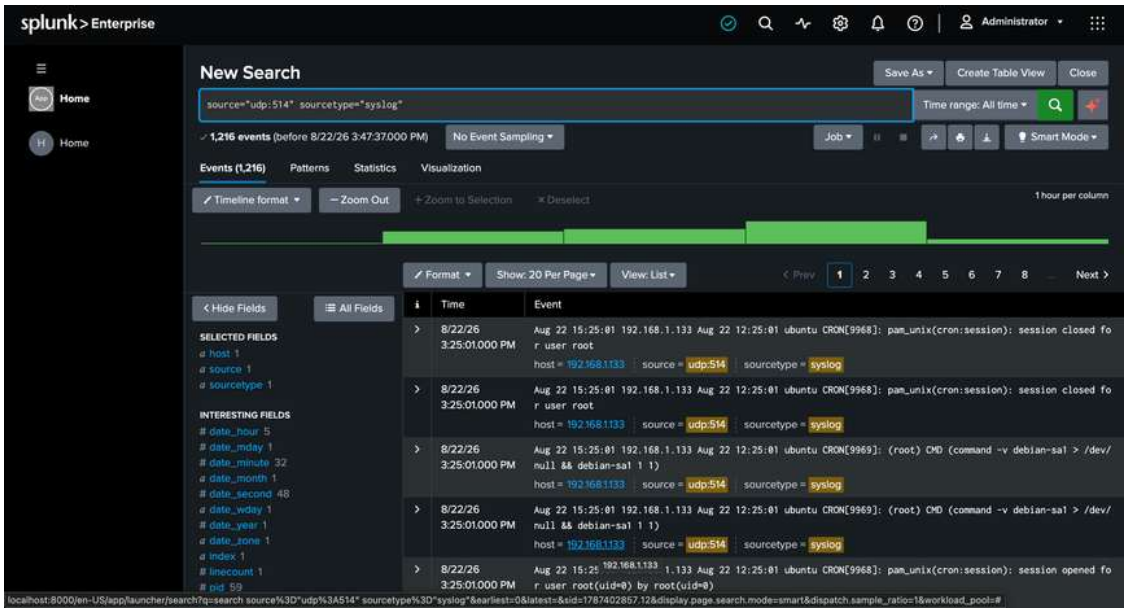


Figure 12 — New Search for source="udp:514" sourcetype="syslog" returning 1,216 indexed events from the Ubuntu target.

5. Attack Simulation

5.1 Attack Objective

The objective of the simulated attack is to obtain valid SSH credentials for the user account "abdelrahman" on the Ubuntu target (192.168.1.133) through automated password guessing, and then to demonstrate what an attacker would typically do immediately after gaining shell access.

5.2 Hydra

Hydra is executed from the Kali attacker machine and is directed at the SSH service running on the Ubuntu target. It automates the process of trying many candidate passwords against a single username in rapid succession, which is the defining behavior of a brute-force / password-spraying style attack that the SOC lab is built to detect.

5.3 Password List

A local wordlist file was created on Kali using the echo command, with candidate passwords separated by newline characters:

```
echo -e "123456\nadmin\npassword\ntest123\nabdo2026\nwrongpass" > passwords.txt
```

This produced an initial six-entry wordlist (123456, admin, password, test123, abdo2026, wrongpass) that did not contain the target account's actual password.

5.4 SSH Authentication Attempts

The attack itself — the repeated SSH login attempts — is performed entirely from Kali using Hydra. Detection and investigation of that activity happen separately and afterward, on the Splunk side, once the resulting logs have been forwarded from Ubuntu. The command used to launch the attack was:

```
hydra -l abdelrahman -P passwords.txt ssh://192.168.1.133 -t 4 -V
```

This instructs Hydra to attack SSH (ssh://) on host 192.168.1.133, using the fixed username abdelrahman (-l), trying every password in passwords.txt (-P), with 4 parallel connection tasks (-t 4), in verbose mode (-V) so every attempt is printed to the console.

Stage 1 — Attack with an incorrect password list

With the initial six-entry wordlist, Hydra attempted all six candidate passwords against the target and reported the result:

```
1 of 1 target completed, 0 valid password found
```

No valid credentials were found in this stage. From a SOC/security standpoint, this represents a blocked brute-force attempt: authentication failures were generated on the target, but no compromise occurred.

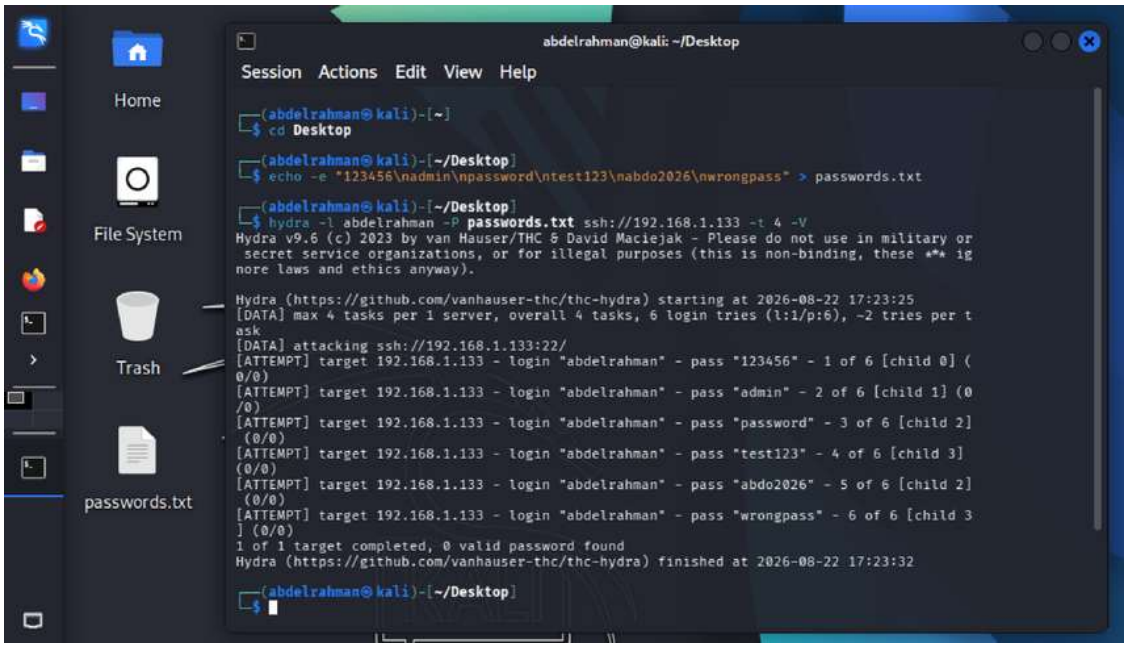


Figure 13 — Kali terminal: first Hydra run against the incomplete wordlist — 0 valid password found.

Stage 2 — Attack with the correct password included

The wordlist was then updated to include the correct password, "123", appended after the original six entries:

```
echo -e "123456\nadmin\npassword\ntest123\nabdo2026\nwrongpass\n123" > passwords.txt
```

Re-running the same Hydra command against the now seven-entry wordlist produced a successful result:

```
[22][ssh] host: 192.168.1.133   login: abdelrahman   password: 123
1 of 1 target successfully completed, 1 valid password found
```

This confirms that the target account "abdelrahman" was successfully compromised using the weak password "123", after six prior failed attempts. This progression — several failed logins immediately followed by a successful one from the same source — is the exact behavioral pattern the SOC detection query is designed to surface.

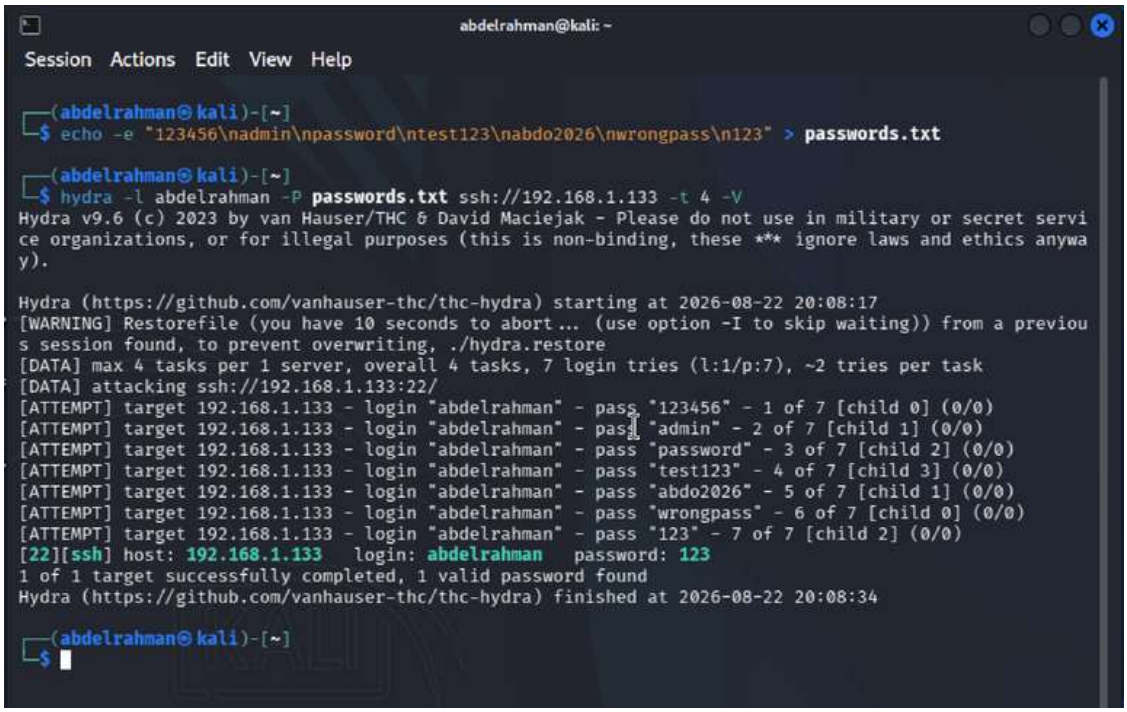


Figure 14 — Kali terminal: second Hydra run with the correct password included — 1 valid password found (abdelrahman / 123).

Post-Compromise Access

With the valid credentials confirmed, the attacker authenticated interactively over SSH from Kali to the Ubuntu target and, once logged in, created a directory to demonstrate post-authentication activity:

```
ssh abdelrahman@192.168.1.133
sudo mkdir testfolder
```

The SSH session banner confirmed a successful login to “Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-138-generic aarch64).” The subsequent sudo mkdir command — and later directory-management commands used during investigation — were captured in the Ubuntu system logs and forwarded to Splunk along with the authentication events.

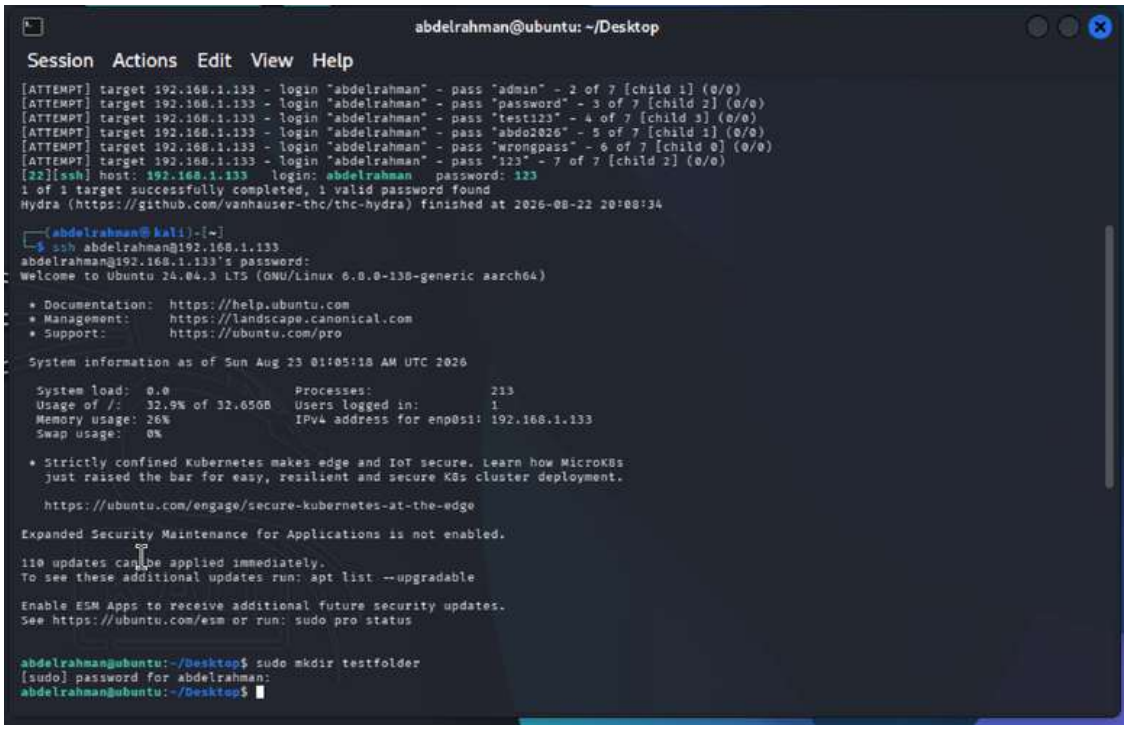


Figure 15 — Interactive SSH login to the compromised account and creation of a test directory (sudo mkdir testfolder) on the Ubuntu target.

6. Log Collection and SIEM Integration

The complete log pipeline connects the attack activity on Kali to the detection and investigation performed in Splunk. Each stage below carries a specific purpose in that chain:

Stage	Purpose
Kali → SSH authentication attempts	Generates the brute-force traffic against the target
Ubuntu → authentication/system logs	The target's OpenSSH/PAM/sudo/cron subsystems record every login attempt and post-login command locally
rsyslog	Reads local log messages and forwards them off-host to Splunk in real time
UDP 514	Carries the forwarded Syslog messages from Ubuntu to Splunk across the network
Splunk (indexing)	Receives and indexes the forwarded events under sourcetype=syslog, source="udp:514"
Search & analysis	SPL queries are run against the indexed data to detect and investigate the incident

Because rsyslog forwards continuously and in real time, the events visible in Splunk (session open/close events, cron command executions, and — once the attack begins — SSH authentication failures and successes) reflect the live state of the Ubuntu target rather than a periodic export, which is what allows the detection query to be run against current activity.

7. Detection and Incident Analysis

7.1 Detection Logic

The detection approach used in this project identifies a source IP address that generates a large number of failed SSH authentication attempts against the target, and separately determines whether that same source went on to authenticate successfully. The specific factors examined are:

- Repeated failed password events ("Failed password for ...") from the same source IP against the same host.
- The attacking source IP, extracted directly from the authentication log message.
- Whether a successful authentication event ("Accepted password for ...") occurs from that same source.
- A minimum failed-attempt threshold of four or more (failed_attempts >= 4) used in the query to isolate genuine brute-force behavior from isolated login mistakes.
- Commands executed via sudo by the account after authentication, used to reconstruct post-compromise activity.

This threshold of four failed attempts is the value actually used in the SPL query documented below; no other threshold was tested or is claimed by the project.

7.2 SPL Queries

Two versions of the detection query were used over the course of the investigation, reflecting its iterative refinement as the incident was investigated further (first to detect the brute-force pattern and successful login, then again after the post-authentication folder activity had occurred). Both are documented below exactly as implemented.

Query 1 — Initial brute-force and successful-login detection

```
index=* sourcetype=syslog ("password for" OR "sudo:" OR "session opened")
| dedup _time _raw
| rex "(?<action>Failed|Accepted) password for (invalid user )?(?<target_user>\S+) from
(?<attacker_ip>\S+)"
| rex "sudo:\s+(?<executor>\S+)\s+:.+;\s+COMMAND=(?<attacker_command>.*)"
| stats
    count(eval(action="Failed")) as failed_attempts,
    count(eval(action="Accepted")) as successful_logins,
    values(attacker_command) as all_executed_actions,
    earliest(_time) as attack_start,
    latest(_time) as attack_end
    by attacker_ip, host
| where failed_attempts >= 4
| eval
    status = if(successful_logins >= 1, "Compromised (Breached)", "Blocked"),
    attack_start = strftime(attack_start, "%H:%M:%S"),
    attack_end = strftime(attack_end, "%H:%M:%S")
| table attack_start attack_end attacker_ip host failed_attempts successful_logins status
all_executed_actions
```

Query breakdown:

- index=* sourcetype=syslog (“password for” OR “sudo:” OR “session opened”) — narrows the search to the forwarded syslog events that are relevant to authentication and privileged command execution.

- dedup _time _raw — removes duplicate events with the same timestamp and raw text, avoiding double counting.
- The first rex extracts action (Failed/Accepted), target_user, and attacker_ip from SSH authentication log lines.
- The second rex extracts executor and attacker_command from sudo log lines, capturing exactly which command was run after authentication.
- stats ... by attacker_ip, host — aggregates, per attacking IP and target host, the count of failed attempts, the count of successful logins, every distinct command executed, and the earliest/latest event timestamps.
- where failed_attempts >= 4 — filters the results down to sources that generated at least four failed attempts, the threshold used to flag brute-force behavior.
- eval status = if(successful_logins >= 1, “Compromised (Breached)”, “Blocked”) — classifies the incident outcome based on whether any successful login occurred.
- table ... — presents the final, analyst-readable summary: when the attack started and ended, who the attacker was, how many attempts failed, whether any succeeded, the resulting status, and every command the attacker ran.

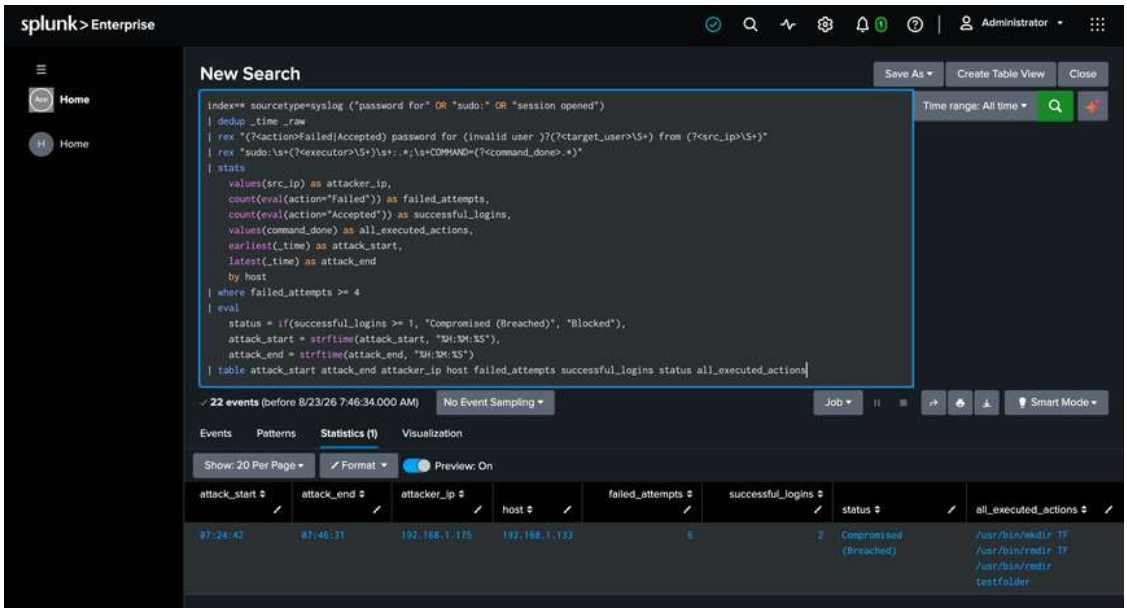


Figure 16 — Result of Query 1: attacker 192.168.1.175 shows 6 failed attempts, 1 successful login, and status “Compromised (Breached).”

Query 2 — Updated query after post-authentication folder activity

After the attacker created and removed test directories on the target following the successful login, the query was re-run with minor field-naming adjustments (src_ip / command_done instead of attacker_ip / attacker_command, and aggregation by host only) to capture the additional sudo activity:

```
index=* sourcetype=syslog ("password for" OR "sudo:" OR "session opened")
| dedup _time _raw
| rex "(?<action>Failed|Accepted) password for (invalid user )?(?<target_user>\S+) from
(?<src_ip>\S+)"
| rex "sudo:\s+(?<executor>\S+)\s+:.+;\s+COMMAND=(?<command_done>.*)"
| stats
    values(src_ip) as attacker_ip,
    count(eval(action="Failed")) as failed_attempts,
    count(eval(action="Accepted")) as successful_logins,
```

```
    values(command_done) as all_executed_actions,
    earliest(_time) as attack_start,
    latest(_time) as attack_end
    by host
| where failed_attempts >= 4
| eval
    status = if(successful_logins >= 1, "Compromised (Breached)", "Blocked"),
    attack_start = strftime(attack_start, "%H:%M:%S"),
    attack_end = strftime(attack_end, "%H:%M:%S")
| table attack_start attack_end attacker_ip host failed_attempts successful_logins status
all_executed_actions
```

The logic is otherwise identical to Query 1: it still flags any host/source pair with four or more failed attempts, still classifies the outcome as Compromised (Breached) once at least one successful login is present, and still lists every command executed by the attacker via sudo — now aggregated per target host rather than per attacker/host pair.

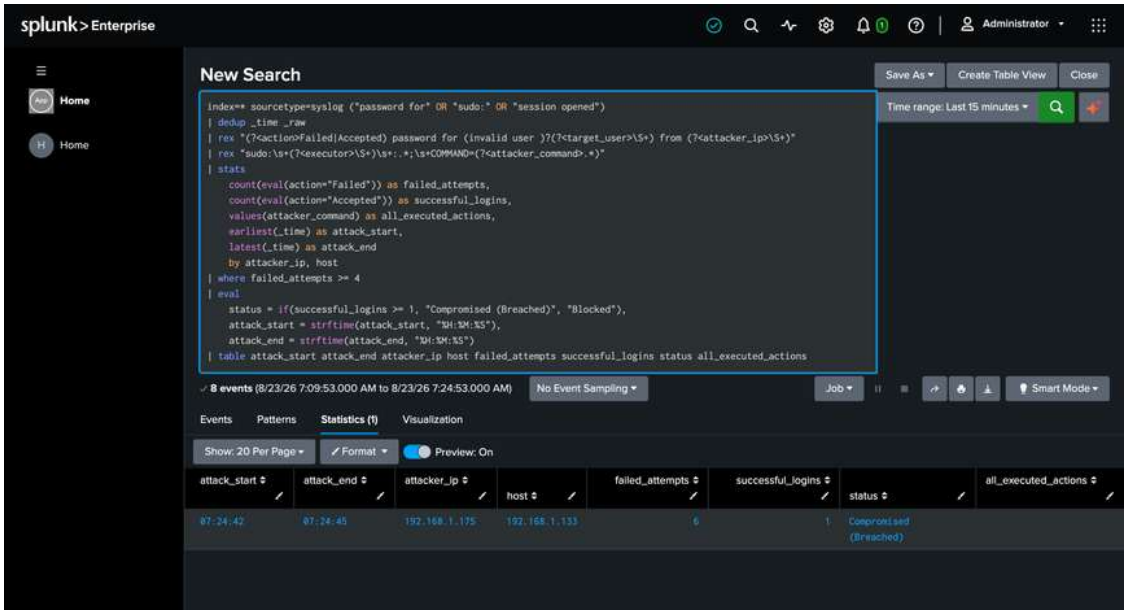


Figure 17 — Result of Query 2 after the attacker created and removed test directories: 22 underlying events, failed_attempts 6, successful_logins 2, and all_executed_actions listing /usr/bin/mkdir and /usr/bin/rmdir for TF and testfolder.

7.3 Successful Authentication

A single failed login is rarely significant on its own, but a successful authentication that follows several failed attempts from the same source materially changes the severity of the event: it is no longer an attempted intrusion but a confirmed account compromise. This is precisely why the query above explicitly separates failed_attempts from successful_logins and derives a status field from the combination of the two, rather than only counting failures. In this project, the source IP 192.168.1.175 (Kali) generated six failed authentication attempts against 192.168.1.133 (Ubuntu) before a successful login was recorded for the same source, which the query correctly classifies as “Compromised (Breached).”

7.4 Post-Authentication Activity

After the attacker's SSH session succeeded, directory-creation and directory-removal commands were executed via sudo on the target and captured through the second rex pattern in the SPL queries above. Across the investigation, the commands observed in the all_executed_actions field included creating and removing

directories such as testfolder and TF (for example /usr/bin/mkdir testfolder, /usr/bin/rmdir testfolder, /usr/bin/mkdir TF, /usr/bin/rmdir TF), performed directly from the compromised account's shell.

Post-authentication activity matters during incident investigation because it establishes what an attacker actually did with the access they obtained, beyond the login itself. Even a simple, low-impact action like creating and deleting a folder demonstrates that the attacker had working, privileged (sudo) command execution on the host — the same command path that could, in a real incident, be used to stage tools, exfiltrate data, or modify the system. Capturing these actions in the same SPL query as the authentication events allows an analyst to see the full scope of an intrusion (access gained → actions taken) in a single, correlated view.

8. Incident Investigation Workflow

The investigation performed in this project followed a standard SOC workflow, moving from initial detection to a documented, visualized conclusion:

1. Identify suspicious authentication activity by searching forwarded syslog events for password-related and sudo-related messages.
2. Examine failed login attempts and count them per source IP using the SPL stats command.
3. Identify the source of the activity (attacker_ip / src_ip = 192.168.1.175) directly from the extracted log fields.
4. Identify successful authentication, if present, using the same query (successful_logins >= 1).
5. Examine activity performed after successful access by extracting sudo COMMAND values into all_executed_actions.
6. Correlate the failed attempts, the successful login, and the post-login commands into a single per-source, per-host summary using stats ... by.
7. Determine whether the behavior represents a security incident using the derived status field (Compromised (Breached) vs Blocked).
8. Visualize the findings by saving the query as a panel on a Splunk dashboard for ongoing monitoring.

9. Splunk Dashboard

The final SPL detection query was saved directly into a new Splunk dashboard so the brute-force detection and post-authentication activity could be reviewed without re-running the search manually each time. The dashboard was created from the Search & Reporting app by selecting Save As > New Dashboard from the search results screen.

Setting	Value
Dashboard Title	Linux Credential Access & Execution
Panel Title	Brute-Force Detection & Executed Actions
Dashboard Type	Classic Dashboard
Visualization Type	Statistics Table

Setting	Value
Permissions	Private

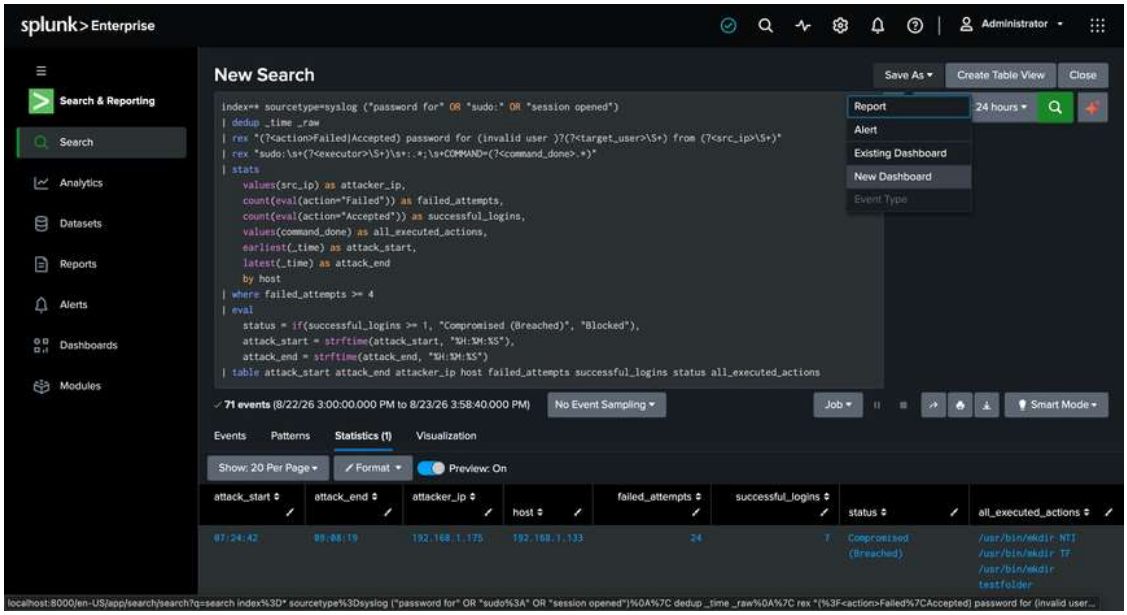


Figure 18 — Saving the detection query as a panel: Save As > New Dashboard from the Search & Reporting results screen.

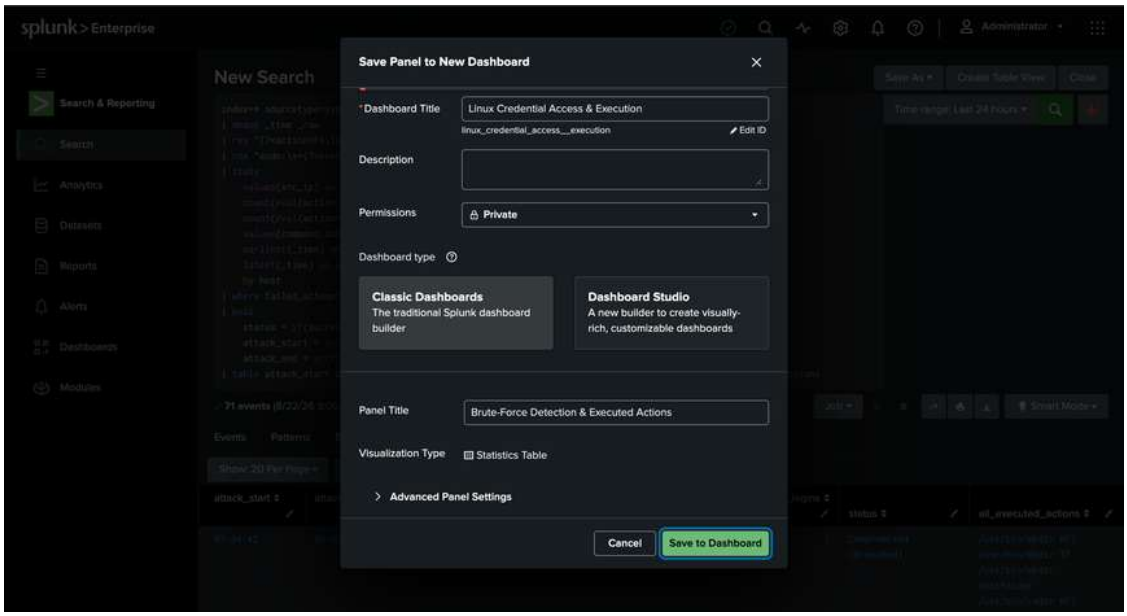


Figure 19 — Save Panel to New Dashboard dialog: Dashboard Title "Linux Credential Access & Execution," Panel Title "Brute-Force Detection & Executed Actions," Classic Dashboard, Statistics Table visualization.

The dashboard panel displays the same summarized fields produced by the detection query: attack_start, attack_end, attacker_ip, host, failed_attempts, successful_logins, status, and all_executed_actions. In its saved state, the dashboard's "Brute-Force Detection & Executed Actions" panel confirmed the incident summary for attacker 192.168.1.175 against host 192.168.1.133, showing multiple failed attempts, one or more successful logins, a status of "Compromised (Breached)," and the post-authentication mkdir/rmdir commands executed by the attacker.

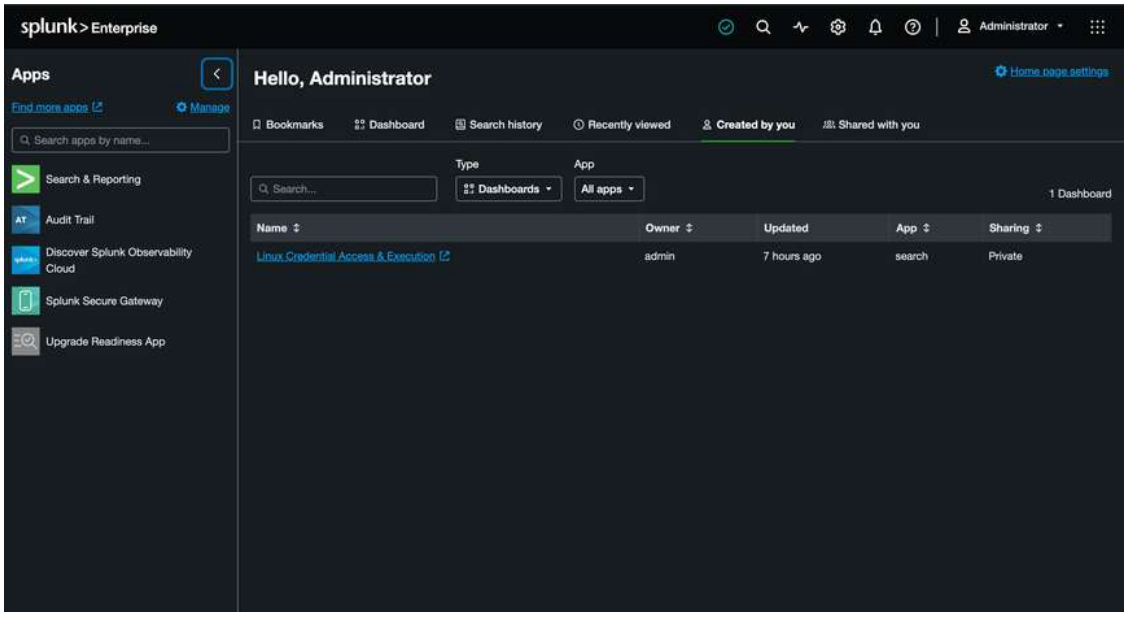


Figure 20 — The saved "Linux Credential Access & Execution" dashboard listed under Apps > Search & Reporting > Created by You.

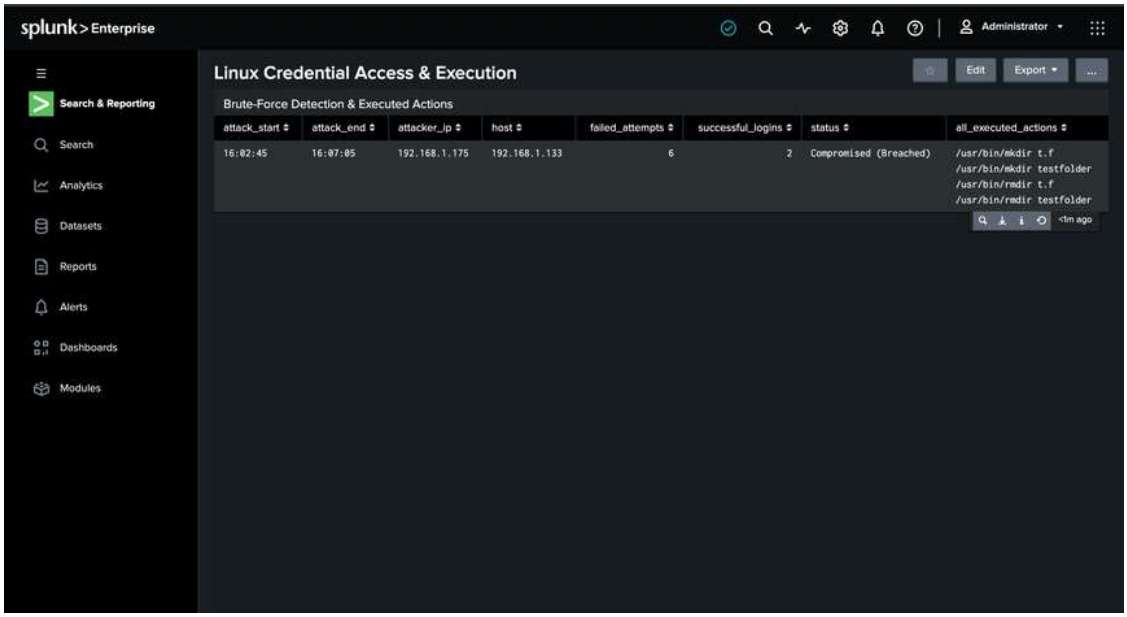


Figure 21 — Final dashboard view showing the Brute-Force Detection & Executed Actions panel with the completed incident summary.

For a SOC analyst, dashboards like this are useful because they convert a manually constructed investigation query into a persistent, reusable monitoring view. Rather than re-typing and re-running the SPL search every time a new brute-force attempt needs to be reviewed, the analyst can open the dashboard directly and see the current status of authentication attacks against the monitored host at a glance, which is essential for maintaining situational awareness across many hosts in a real production SOC.

10. Challenges and Solutions

The project source material (the SOC lab documentation and screenshots) does not document specific technical difficulties encountered while building the lab — for example, configuration errors, connectivity failures, or query

debugging steps — beyond the attack progression itself. The only documented "iterative correction" visible in the source is the brute-force attempt itself: the first Hydra run used a wordlist that did not contain the correct password and returned zero valid credentials, and the wordlist was subsequently updated to include the correct password, after which the attack succeeded. This is presented in Section 5.4 as part of the attack simulation rather than as a lab-building challenge, since it reflects the intended two-stage demonstration rather than an unplanned obstacle.

If specific setup or troubleshooting challenges were encountered while configuring rsyslog forwarding, the Splunk UDP input, or the SPL queries, that information was not included in the provided source material. Additional project notes describing those difficulties, and how they were resolved, would be needed to populate this section further.

11. Testing and Validation

Validation was performed at each stage of the pipeline, confirmed directly through the Splunk interface and terminal output captured in the project:

- Log forwarding verification — after configuring rsyslog and the Splunk UDP input, a search for source="udp:514" sourcetype="syslog" returned 1,216 indexed events, confirming that logs generated on Ubuntu were being received by Splunk.
- Splunk ingestion verification — the Data Inputs > UDP configuration screen confirmed the input was created successfully with Port 514 and Source Type syslog, and the platform displayed the "UDP input has been created successfully" confirmation message.
- Brute-force simulation verification (negative case) — the first Hydra run against the incorrect password list completed with the explicit result "0 valid password found," confirming the attack was correctly blocked when credentials were wrong.
- Brute-force simulation verification (positive case) — the second Hydra run, with the correct password included, completed with the explicit result "1 valid password found" and reported the exact credential pair (abdelrahman / 123).
- Detection query verification — the SPL detection query was run and re-run at multiple points as the investigation progressed (returning 8, then 22, then 71 underlying events at different stages), each time correctly aggregating failed_attempts, successful_logins, and status for the attacking source.
- Successful login verification — the interactive SSH session banner ("Welcome to Ubuntu 24.04.3 LTS") confirmed that the credentials obtained via Hydra worked for an actual interactive login, not just the Hydra test connection.
- Post-authentication activity verification — the sudo mkdir/rmdir commands run interactively on Ubuntu appeared correctly in the all_executed_actions field of the SPL query output in Splunk, confirming the sudo-command extraction (rex) worked as intended.
- Dashboard verification — after saving the panel, the dashboard "Linux Credential Access & Execution" appeared under Apps > Search & Reporting > Created by You, and opening it displayed the same investigation summary produced by the underlying search.

No numerical performance metrics (such as detection latency or throughput) were measured or are claimed by the project; the validation above is functional/behavioral, confirming that each stage of the pipeline worked as configured.

12. Results and Findings

Technical Results

- A working, centralized log pipeline was established from an Ubuntu SSH server to Splunk using rsyslog and a UDP Syslog input on port 514.
- Splunk successfully indexed forwarded authentication, session, and sudo command events under sourcetype=syslog.
- Two SPL detection queries were built and validated, correctly extracting attacker IP, failed/successful login counts, timing, and executed sudo commands from raw log text using rex.
- A Classic Dashboard ("Linux Credential Access & Execution") was created with a Statistics Table panel that persists the detection query for ongoing use.

Security Results

- A simulated SSH brute-force attack from 192.168.1.175 against 192.168.1.133 was detected based on repeated failed authentication attempts (failed_attempts >= 4) from a single source.
- The attack's outcome was correctly classified: blocked in the first stage (wrong password list) and "Compromised (Breached)" in the second stage, once the correct password ("123") was used successfully by the account "abdelrahman."
- Post-authentication actions taken by the attacker (directory creation/removal via sudo) were captured and linked to the same incident record.

SOC Value

This workflow gives a SOC analyst a single, correlated view that answers the three questions that matter most in an authentication-based incident: who attempted access, whether they succeeded, and what they did afterward. By centralizing the logs and pre-building the correlation logic into a saved SPL query and dashboard, the time between an attack occurring and an analyst recognizing it as a confirmed compromise is significantly reduced compared to reviewing raw, host-local log files manually.

13. Security Analysis

In general, brute-force attacks against authentication services like SSH are dangerous because they require little sophistication or infrastructure from the attacker, can be fully automated, and are effective against any account protected only by a weak or common password. As a recommended practice, authentication logs are considered one of the highest-value log sources in any environment because they are the primary record of who is attempting to access a system and whether that access succeeded — without them, an organization has no reliable way to distinguish a failed intrusion attempt from a successful one.

From a SOC perspective, centralized log collection is important because attacker activity that is only recorded in a local file on the target host is easily missed, deleted, or simply never reviewed. Forwarding logs to a central SIEM, as demonstrated in this project, ensures that evidence of an attack survives independently of the compromised host and can be searched alongside data from other systems. SIEM correlation and search (SPL, in this case) is what turns large volumes of raw log lines into an actionable summary — the manual equivalent of the queries used in Section 7 would require an analyst to read through thousands of individual log lines by hand.

As documented in the project, successful authentication following multiple failed attempts is a decisive severity indicator: it converts an “attempted” event into a “confirmed compromise” event, which typically triggers a different, more urgent incident-response process (containment, credential rotation, forensic review) than a blocked attack would. Likewise, as a recommended practice, post-authentication activity should always be investigated once a compromise is confirmed, because the login itself only establishes that the attacker gained a foothold — the commands executed afterward determine the actual impact of the intrusion, whether that is reconnaissance, data access, persistence, or (as demonstrated at a small scale in this lab) simple file-system changes.

14. Future Improvements

The following enhancements are proposed as future improvements to this lab. None of these are currently implemented; they are recommendations for extending the project.

- Automated alerting — configure a Splunk alert to trigger on the existing detection query (e.g., whenever status = "Compromised (Breached)") instead of requiring an analyst to run the search manually.
- Threshold-based detection tuning — evaluate and adjust the failed_attempts >= 4 threshold against a larger dataset to reduce false positives/negatives.
- Correlation rules — extend the SPL logic to correlate brute-force detections with other event types (e.g., new user creation, privilege escalation) for broader attack-chain visibility.
- Automated incident response — integrate the detection with a response action, such as automatically disabling the compromised account or triggering a firewall rule.
- IP blocking — add an automated or semi-automated step to block the attacker_ip at the network or host firewall level once a source is confirmed compromised.
- Enhanced dashboards — expand the current Statistics Table panel with time-series visualizations (e.g., failed-login trend over time) and additional panels per host.
- Risk scoring — assign a numeric risk score to each detected source based on failed-attempt volume, success status, and post-authentication activity.
- MITRE ATT&CK mapping — tag the detection with relevant techniques, such as T1110 (Brute Force) and T1078 (Valid Accounts), to align the lab with a standard threat framework.
- Additional log sources — forward additional Ubuntu log sources (e.g., full auth.log detail, file-integrity monitoring) to enrich the investigation beyond authentication and sudo events.
- Authentication monitoring at scale — extend the lab to multiple target hosts to demonstrate cross-host brute-force / credential-stuffing detection.
- SOAR integration — connect the Splunk detection to a SOAR platform to automate the investigation and response workflow described in Section 8.

15. Conclusion

This project built and validated a complete, practical SOC lab for automated brute-force detection and incident analysis. A three-machine environment — Kali Linux as the attacker, Ubuntu Server as the target, and Splunk Enterprise as the SIEM — was configured so that authentication and system logs generated on the target were forwarded in real time via rsyslog over Syslog/UDP to a centrally managed Splunk instance.

A realistic SSH brute-force attack was simulated using Hydra, first with an incorrect password list to demonstrate a blocked attack, and then with an updated list containing the correct credential to demonstrate a successful compromise of the account “abdelrahman” using the weak password “123.” The resulting logs were collected in Splunk and analyzed using purpose-built SPL queries that extracted the attacking IP, counted failed and successful authentication events, and captured the commands executed by the attacker after gaining access, including the creation and removal of test directories.

The investigation was carried through to a documented conclusion: the source 192.168.1.175 was identified, its authentication activity against 192.168.1.133 was classified as “Compromised (Breached)” based on failed-attempt volume and a confirmed successful login, and its post-authentication actions were recorded. The findings were then pinned to a Splunk dashboard, “Linux Credential Access & Execution,” for ongoing visualization.

From a SOC perspective, this project demonstrates the core value of log centralization and SIEM correlation: turning isolated, host-local authentication events into a single, actionable incident record that answers who attacked, whether they succeeded, and what they did next — the same fundamental workflow used in production SOC environments to detect and investigate real credential-based attacks.

16. References

The following resources are directly relevant to the tools and technologies used in this project, based on the project source material:

- THC-Hydra — the SSH brute-force tool used to simulate the attack; project repository referenced directly in the tool's own console output: https://github.com/vanhauser-thc/thc-hydra
- Splunk Enterprise — the SIEM platform used for log indexing, SPL search, and dashboarding.
- rsyslog — the log-forwarding daemon used on the Ubuntu target.
- OpenSSH (openssh-server) — the SSH service configured and targeted on the Ubuntu host.
- Ubuntu Server 24.04 LTS — the operating system of the target machine.
- Kali Linux — the penetration-testing distribution used as the attacker platform.

The source material provided for this documentation did not include a formal bibliography or additional external citations. If specific product documentation pages, academic references, or training-program materials (e.g., from NTI, EG-CERT, or NTRA) should be cited, they should be added here manually with their full URLs.